

INSTITUTO TECNOLÓGICO DE BUENOS AIRES

Procesamiento Avanzado de Imágenes en Biología y Biomedicina (16.85)

Trabajo Práctico Final

Karyomatic:

Cariotipado automático de cromosomas por
segmentación de instancias y registración
mediante aprendizaje profundo

Santiago Gruss 63465

Agustín Miguel 63214

Docente: Roberto Sebastián Tomás

Resumen

El cariotipado (detectar, clasificar y ordenar los cromosomas de una célula en metafase) es un procedimiento de rutina en citogenética clínica, hoy manual y lento. En este trabajo se desarrolla un pipeline automático sobre el dataset público AutoKary2022, integrando preprocesamiento (denoising Non-Local Means y realce de contraste CLAHE), segmentación de instancias con Mask R-CNN, registración de cada cromosoma a una pose canónica mediante PCA, clasificación con una red VGG16 con atención Squeeze-and-Excitation, y ensamblado del cariograma con conteo por par y marcado de anomalías. La segmentación alcanza un AP@0.5 del 97,8 % sobre el conjunto de test y la clasificación una exactitud del 87 % sobre las 24 clases cromosómicas. Se presenta además una interfaz gráfica en Streamlit que ejecuta y visualiza cada etapa.

Índice

Resumen	1
1. Introducción	2
1.1. Marco teórico	2
1.2. Estado del arte	2
1.3. Objetivos	2
2. Materiales y Métodos	3
2.1. Datos: AutoKary2022	3
2.2. Arquitectura general del pipeline	3
2.3. Caracterización de la imagen y estudio de preprocesamiento	3
2.4. Segmentación de instancias (Mask R-CNN)	4
2.5. Extracción de características y registración	4
2.6. Clasificación	5
2.7. Ensamblado del cariograma	6
2.8. Interfaz gráfica	6
3. Resultados	7
3.1. Caracterización del preprocesamiento	7
3.2. Segmentación	8
3.3. Extracción y registración	8
3.4. Clasificación	8
3.5. Cariograma	9
4. Conclusiones	10
A. Anexo: detalle de la clasificación	12

1. Introducción

1.1. Marco teórico

Una célula humana somática sana contiene 23 pares de cromosomas: 22 pares de autosomas y un par de cromosomas sexuales (XX en la mujer, XY en el hombre) [1]. El cariotipo es la representación ordenada de esos cromosomas, agrupados por pares y dispuestos por tamaño y por la posición del centrómero. Su análisis permite detectar anomalías numéricas (por ejemplo, la trisomía 21 asociada al síndrome de Down) y estructurales, y constituye un examen de referencia para el diagnóstico de trastornos genéticos, malformaciones congénitas y ciertos cánceres [2].

La construcción del cariotipo parte de una imagen microscópica de una célula detenida en metafase, etapa del ciclo celular en la que los cromosomas están máximamente condensados y son individualmente distinguibles. Las células se tiñen con la técnica de bandeado G (Giemsa), que produce un patrón característico de bandas claras y oscuras a lo largo de cada cromosoma. A partir de esa imagen, un citogenetista debe: (i) identificar cada cromosoma como un objeto individual, incluso cuando se tocan o se superponen; (ii) clasificarlo en una de las 24 clases posibles (1–22, X, Y); y (iii) ordenarlos en la grilla estándar del cariograma. Este proceso manual insuere entre 30 y 50 minutos por caso y depende de personal altamente especializado [1,2], lo que motiva su automatización.

1.2. Estado del arte

El cariotipado automático tiene una larga historia, pero se ha revitalizado con el aprendizaje profundo. Un obstáculo recurrente ha sido la escasez de conjuntos de datos públicos, densamente anotados y clínicamente realistas. You *et al.* [1] abordan esa carencia con AutoKary2022, un dataset de más de 27000 instancias de cromosomas anotadas con máscara poligonal en 612 imágenes microscópicas de 50 pacientes, pensado específicamente para segmentación de instancias. Sobre él evalúan métodos representativos y muestran que las morfologías complejas (cromosomas alargados, curvados, en contacto o superpuestos) hacen del problema un desafío abierto.

En cuanto a los flujos completos, Wang *et al.* [2] proponen una arquitectura integrada de cariotipado totalmente automático que separa segmentación (módulo *ChrRender*, basado en Mask R-CNN) y clasificación (módulo *ChrNet4*). Reportan, en la tarea separada, un AP@0.5 de segmentación de 96,65 % (dataset público BioImLab) y 96,81 % (dataset privado), y una exactitud de clasificación en el rango de 94 %–95 %. Estos valores constituyen la referencia de estado del arte contra la que se contrastan los resultados de este trabajo (Sección 4).

La elección de arquitecturas para la segmentación sigue las líneas dominantes en visión por computadora: Mask R-CNN [3] con *backbone* ResNet-50 [4] y *Feature Pyramid Network* [5]. Para la clasificación se recurre a redes convolucionales preentrenadas en ImageNet [6] ajustadas por *transfer learning*, como VGG16 [7], opcionalmente enriquecidas con mecanismos de atención por canal [8].

1.3. Objetivos

El objetivo general es desarrollar y evaluar un pipeline automático de cariotipado que, partiendo de una imagen de metafase, segmente, rectifique y clasifique cada cromosoma para ensamblar el cariograma final, e integre una interfaz gráfica de visualización. Los objetivos específicos son:

1. Caracterizar y aplicar una etapa de pre-procesamiento (estimación de ruido, denoising y realce de contraste) adecuada a imágenes de bandeado G, evaluada cuantitativamente.
2. Entrenar y evaluar un modelo de segmentación de instancias de cromosomas sobre AutoKary2022, reportando métricas estándar (AP a distintos umbrales de IoU).

3. Implementar la extracción y registración de cada cromosoma a una pose canónica mediante su eje principal (PCA).
4. Clasificar cada cromosoma segmentado en su tipo (1–22, X, Y) mediante una red neuronal convolucional.
5. Desarrollar una interfaz gráfica que ejecute el pipeline y visualice cada etapa.
6. Contrastar los resultados obtenidos con el estado del arte.

2. Materiales y Métodos

2.1. Datos: AutoKary2022

Se utilizó el dataset público AutoKary2022 [1], compuesto por 612 imágenes microscópicas de células en metafase (tinción Giemsa) provenientes de 50 pacientes, con más de 27000 instancias de cromosomas anotadas manualmente. Cada instancia incluye una máscara poligonal y una etiqueta de clase. Las anotaciones, originalmente en formato LabelMe (un archivo JSON por imagen), se convirtieron al formato COCO [9], estándar para detección y segmentación, consolidando todas las anotaciones en un único JSON por partición (`annotations_train.json` / `annotations_test.json`). Todo el pipeline trabaja con imágenes estandarizadas a 1600×1600 píxeles.

2.2. Arquitectura general del pipeline

El pipeline de cariotipado encadena las siguientes etapas, desde la imagen de metafase hasta el cariógrama final:

$$\text{imagen cruda} \rightarrow \underbrace{\text{segment.}}_{\text{Mask R-CNN}} \rightarrow \underbrace{\text{extr. + reg.}}_{\text{PCA}} \rightarrow \underbrace{\text{clasif.}}_{\text{CNN}} \rightarrow \text{cariograma}$$

Los modelos operan sobre la imagen cruda; el preprocesamiento se trata como una etapa de caracterización de la imagen (Sección 2.3), fuera de la cadena de inferencia.

2.3. Caracterización de la imagen y estudio de preprocesamiento

El preprocesamiento (módulo `preprocessing.py`) se aborda como una etapa de caracterización de la imagen se estima el ruido y se aplican denoising y realce de contraste, midiendo su efecto de forma cuantitativa (Sección 3). Consta de tres pasos aplicados en orden: conversión a escala de grises, denoising y realce de contraste local.

Estimación de ruido. Para cuantificar el nivel de ruido antes y después del filtrado se emplea el método de Immerkær [10], que estima el desvío estándar del ruido σ convolucionando la imagen con una máscara laplaciana y promediando la respuesta absoluta:

$$\sigma = \frac{\sqrt{\pi/2}}{6(W-2)(H-2)} \sum_{x,y} |I(x,y) * L|, \quad L = \begin{bmatrix} 1 & -2 & 1 \\ -2 & 4 & -2 \\ 1 & -2 & 1 \end{bmatrix}. \quad (1)$$

Denoising. Se utiliza *Non-Local Means* (NLM) [11], que promedia píxeles con vecindarios de intensidad similar en toda la imagen y preserva bordes y texturas, algo clave para no borrar el bandeo G. Se fijó una intensidad de filtrado $h = 7$: las imágenes de AutoKary2022 presentan muy poco ruido, por lo que un filtrado más agresivo eliminaría detalle diagnóstico útil.

Realce de contraste. Se aplica *Contrast Limited Adaptive Histogram Equalization* (CLAHE) [12] sobre el canal de luminancia, con un límite de recorte bajo (`clipLimit = 1.5`). Un realce más fuerte amplificaba el fondo uniforme en manchas; el valor elegido realza el bandeo sin ensuciar el fondo. No se aplica normalización global de intensidad, ya que estiraba el rango dinámico y amplificaba el fondo.

Decisión: los modelos operan sobre la imagen cruda. La razón surge de la caracterización de la imagen (Sección 3): las imágenes de AutoKary2022 son microscopía Giemsa profesional con muy poco ruido, de modo que el denoising tiene poco por corregir; y un realce de contraste agresivo tiende a deteriorar el bandeo G, justamente la característica fina que la segmentación y la clasificación necesitan preservar. Como el preprocesamiento no ofrece margen de mejora pero sí riesgo de degradar esa información, los modelos se entrenan y ejecutan sobre la imagen cruda. El preprocesamiento se conserva como herramienta de visualización a fines de comparar la imagen original y la realzada para observar el ruido estimado antes y después.

2.4. Segmentación de instancias (Mask R-CNN)

La segmentación se resolvió con Mask R-CNN [3] y *backbone* ResNet-50 [4] con *Feature Pyramid Network* [5], implementado sobre Detectron2 [13] (PyTorch [14]). El modelo se inicializó con pesos preentrenados en COCO (configuración `mask_rcnn_R_50_FPN_3x`) y se ajustó a una única clase (`chromosome`): el objetivo de esta etapa es separar cada cromosoma como instancia, sin distinguir aún su tipo.

Los hiperparámetros de entrenamiento se resumen en la Tabla 1: entrada 1600×1600 , optimizador Adam [15] con tasa de aprendizaje 10^{-4} , batch de 1 imagen, 27400 iteraciones (≈ 50 epochs) con 1000 iteraciones de warmup y sin decaimiento de la tasa de aprendizaje. La evaluación se realizó con el protocolo COCO (`COCOEvaluator`) sobre la partición de test.

Cuadro 1: Configuración de entrenamiento del modelo de segmentación.

Parámetro	Valor
Arquitectura	Mask R-CNN, ResNet-50 + FPN
Inicialización	Preentrenado en COCO
Categorías	1 (<code>chromosome</code>)
Tamaño de entrada	1600×1600
Optimizador	Adam, $lr = 10^{-4}$
Batch size	1
Iteraciones	27400 (≈ 50 epochs)
Warmup	1000 iteraciones
Umbral de confianza	0.5

Para su uso en la interfaz, el modelo entrenado (`model_final.pth`) se exportó a TorchScript mediante el trazado del grafo en CPU, verificando la paridad de resultados frente al predictor de Detectron2. Esto permite ejecutar la segmentación con sólo la librería `torch`, sin necesidad de instalar Detectron2 (que no es instalable en el entorno local por falta de compilador y GPU).

2.5. Extracción de características y registración

A partir de cada máscara predicha, esta etapa (módulo `extraction.py`) produce un recorte individual normalizado a 224×224 píxeles. El procedimiento, por cada instancia, es:

1. Se extrae el contorno de la máscara y se calcula su eje principal por PCA [16]: se toma el autovector asociado al mayor autovalor de la matriz de covarianza de los puntos del

contorno. Su dirección define la orientación del cromosoma y la longitud de la proyección sobre él estima su tamaño.

2. Se recorta el cromosoma según su bounding box, aislándolo del fondo con la máscara (fondo blanco).
3. Se rota a orientación vertical según el ángulo del eje principal. Esta rotación a una plantilla canónica común constituye la registración: todos los cromosomas quedan en la misma pose, eliminando la variabilidad por orientación.
4. Se escala preservando el tamaño relativo entre cromosomas (el escalado es global respecto del cromosoma más largo de la imagen) y se centra en un lienzo de 224×224 .

Los recortes se generan sobre la imagen cruda y tomando los puntos desde la máscara predicha, de modo que la salida de esta etapa quede estandarizada para las etapas posteriores del pipeline.

2.6. Clasificación

Cada recorte de 224×224 que produce la etapa anterior se clasifica en una de las 24 clases cromosómicas (1–22, X, Y) mediante una red neuronal convolucional (CNN) basada en *transfer learning*.

Datos de entrenamiento. El clasificador se entrenó sobre recortes individuales generados con el mismo procedimiento de registración de la Sección 2.5 (aislamiento por máscara sobre fondo blanco, rectificación del eje principal por PCA, escala global y centrado a 224×224), aplicado en este caso sobre las máscaras de referencia de AutoKary2022. Se obtuvieron 23 438 recortes de entrenamiento y 2874 de test, organizados en 24 carpetas (una por clase). Las imágenes se mantienen en RGB (las redes preentrenadas esperan tres canales) y la única operación sobre intensidades es el reescalado a $[0, 1]$ que aplica el generador de datos (`ImageDataGenerator`) en tiempo de entrenamiento; no se aplicó normalización adicional de intensidad ni *data augmentation*.

Arquitectura y entrenamiento. Se parte de una red preentrenada en ImageNet [6] sin su cabeza de clasificación (`include_top=False`), con la base congelada salvo sus últimas capas convolucionales, que se ajustan (*fine-tuning*). Sobre la base se agrega una cabeza común: *global average pooling*, una capa densa de 256 unidades (ReLU), *dropout* de 0.5 y una capa *softmax* de 24 salidas. Se optimiza con Adam [15] ($lr = 10^{-4}$) y entropía cruzada categórica, con los *callbacks* `EarlyStopping` (restaurando los mejores pesos) y `ReduceLROnPlateau` (reduce la tasa de aprendizaje al estancarse la pérdida de validación). El entrenamiento se realizó de forma local en CPU.

Modelos explorados. Se compararon dos *backbones* (ResNet-50 [4] y VGG16 [7]) y, sobre el mejor de ellos, la incorporación de un bloque de atención por canal *Squeeze-and-Excitation* (SE) [8] y de estrategias de regularización y balanceo (Tabla 2). El bloque SE recalibra los canales de la última capa convolucional: los resume con *global average pooling*, aprende un peso por canal con dos capas densas (reducción con ReLU y expansión con *sigmoide*) y reescala el tensor original con esos pesos, resaltando los canales más informativos. El modelo final combina VGG16, bloque SE, ponderación de clases (`class_weight` balanceado, que compensa el desbalance de la clase minoritaria), *label smoothing* de 0.1 y regularización L_2 (10^{-4}) en la capa densa.

Cuadro 2: Diferencias metodológicas entre los clasificadores comparados.

Aspecto	ResNet-50	VGG16 base	VGG16+SE v1	VGG16+SE v2	VGG16 final
Backbone	ResNet-50	VGG16	VGG16	VGG16	VGG16
Capas descongeladas	últimas 10	últimas 4	últimas 4	últimas 4	últimas 4
Atención SE	no	no	sí	sí	sí
Regularización L_2	no	no	no	no	sí (10^{-4})
<code>class_weight</code>	no	no	no	no	sí (balanced)
<i>Label smoothing</i>	no	no	no	no	sí (0.1)
Adam (<i>lr</i>)	10^{-4}	10^{-4}	10^{-4}	10^{-4}	10^{-4}
<i>Batch size</i>	5	4	4	4	4
Épocas (tope)	15	40	40	40	40

2.7. Ensamblado del cariograma

A partir de la clase asignada a cada cromosoma, esta etapa (módulo `karyogram.py`) los ordena en la grilla estándar del cariograma. El procedimiento es:

1. **Mapeo de clases.** La salida del clasificador es un índice 0–23 que no coincide con el número de cromosoma, porque el generador de datos de Keras (`flow_from_directory`) ordena las clases alfabéticamente como cadenas ('1','10','11',..., '2','20',...). Se aplica el mapeo inverso exacto para recuperar el número real, con las clases 23 y 24 asignadas a X e Y.
2. **Agrupamiento y ordenamiento.** Los cromosomas se agrupan por clase y se colocan en la grilla estándar: autosomas 1–22 por tamaño decreciente, seguidos de los cromosomas sexuales X e Y. Los homólogos de cada par se ordenan por tamaño para una disposición prolija.
3. **Conteo y detección de anomalías.** Se cuentan las instancias por clase y se marca como posible anomalía numérica todo autosoma cuyo conteo sea distinto de dos (el conteo de X/Y depende del sexo y no se marca por defecto).

El resultado es una imagen única con el cariograma ordenado, la etiqueta y el conteo de cada clase, y el resaltado de las anomalías detectadas.

2.8. Interfaz gráfica

Se desarrolló una interfaz gráfica en Streamlit [17] (Figura 1) que ejecuta el pipeline y visualiza cada etapa del procesamiento. Se eligió Streamlit porque permite mostrar el procesamiento paso a paso (columnas, imágenes, deslizadores) con muy poco código, lo que se ajusta al requisito de visualizar el procesamiento realizado.

El funcionamiento es el siguiente. El usuario sube una imagen de metafase, que se estandariza a 1600×1600 . La barra lateral permite configurar los parámetros de pre-procesamiento (método e intensidad de denoising, límite de CLAHE) y el umbral de confianza de la segmentación. La interfaz muestra entonces, de forma secuencial:

1. **Pre-procesamiento:** original vs. pre-procesada, con el ruido estimado σ antes y después.
2. **Segmentación:** superposición de las máscaras detectadas sobre la imagen, con el conteo de cromosomas. Corre el modelo TorchScript.
3. **Extracción y rectificación:** galería de los cromosomas recortados, rotados a vertical y escalados a 224×224 .
4. **Clasificación:** cada recorte etiquetado con su tipo (1–22, X, Y) y la confianza del modelo. Corre el clasificador exportado a ONNX.
5. **Cariograma:** ensamblado ordenado en la grilla estándar, con el conteo por par y el aviso de posibles anomalías numéricas.

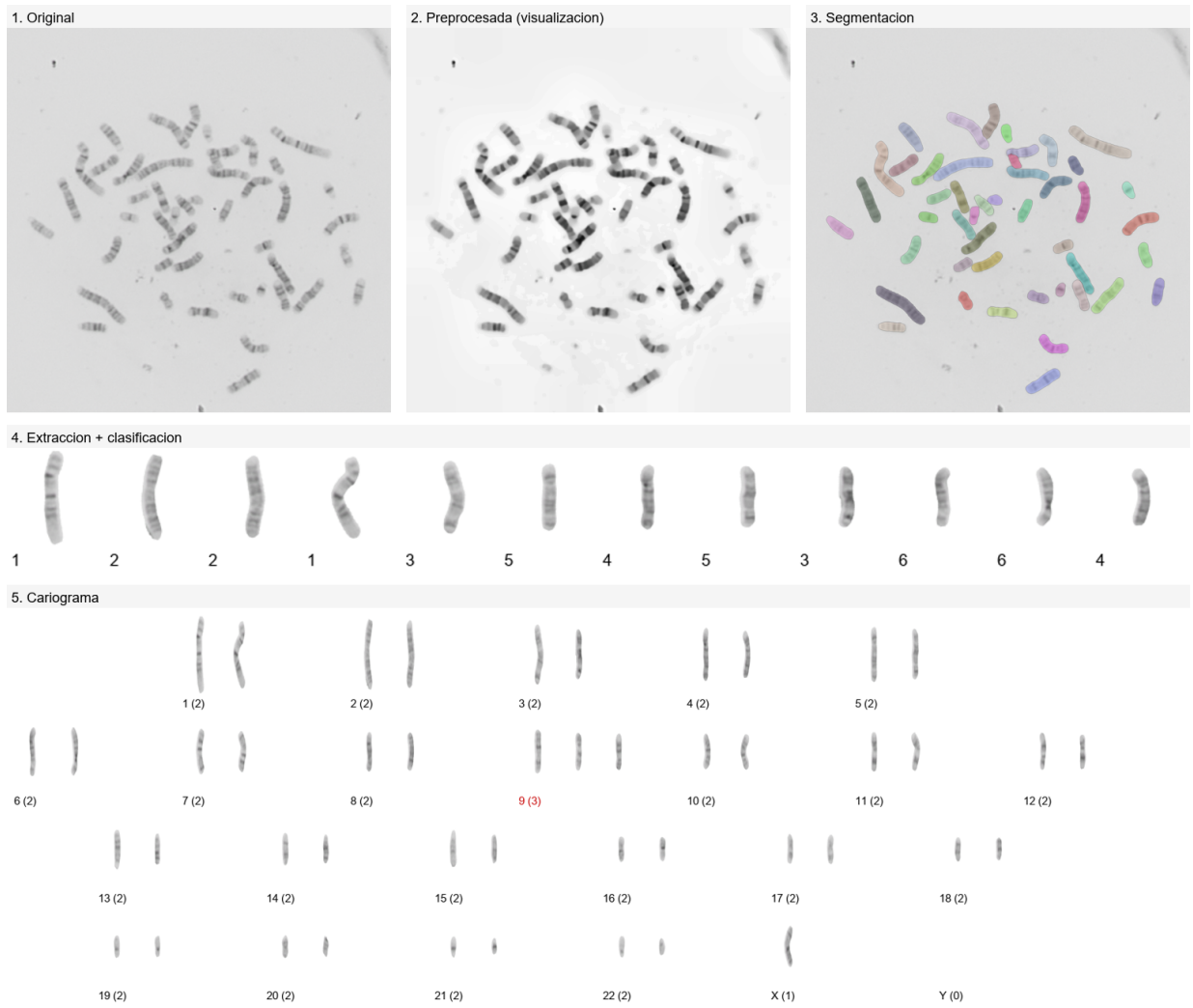


Figura 1: Etapas del procesamiento tal como las presenta la interfaz sobre una metafase de test: original, preprocesada (visualización), segmentación, extracción con la clase predicha de cada cromosoma, y cariógrama ensamblado.

3. Resultados

3.1. Caracterización del preprocesamiento

Sobre una muestra del conjunto de test se cuantificó el efecto del preprocesamiento canónico (NLM con $h = 7$ + CLAHE con `clip` = 1.5). El ruido estimado por el método de Immerkær es muy bajo ya en la imagen original y apenas disminuye con el filtrado ($\sigma \approx 0,7 \rightarrow 0,2$ en promedio): las imágenes parten prácticamente limpias, por lo que el denoising tiene poco por corregir. El SSIM [18] entre la imagen original en escala de grises y la preprocesada es alto ($\approx 0,95$) y el PSNR moderado (≈ 21 dB) refleja el cambio de contraste que introduce CLAHE. El problema es que ese realce, sobre todo con parámetros más agresivos, tiende a atenuar el bandeo G, que es justamente la característica fina de la que dependen la segmentación y la clasificación. Como no hay margen de mejora pero sí riesgo de deteriorar esa información, los modelos se ejecutan sobre la imagen cruda (Sección 2.3).

3.2. Segmentación

El modelo de segmentación se evaluó con el protocolo COCO sobre la partición de test. La Tabla 3 reporta la *Average Precision* (AP) a distintos umbrales de IoU, tanto para las cajas (*bbox*) como para las máscaras (*segm*). El modelo alcanza un **AP@0.5 de 97,8 %** en cajas y **97,9 %** en máscaras, con un AP@0.75 superior al 94 %, lo que indica una segmentación precisa incluso con criterios de solapamiento exigentes.

Cuadro 3: Métricas de segmentación (protocolo COCO) sobre el conjunto de test de AutoKary2022. AP promediado en IoU 0,50:0,95.

Tipo	AP	AP@0.5	AP@0.75
Detección (<i>bbox</i>)	81.5	97.8	94.3
Segmentación (<i>segm</i>)	78.9	97.9	94.0

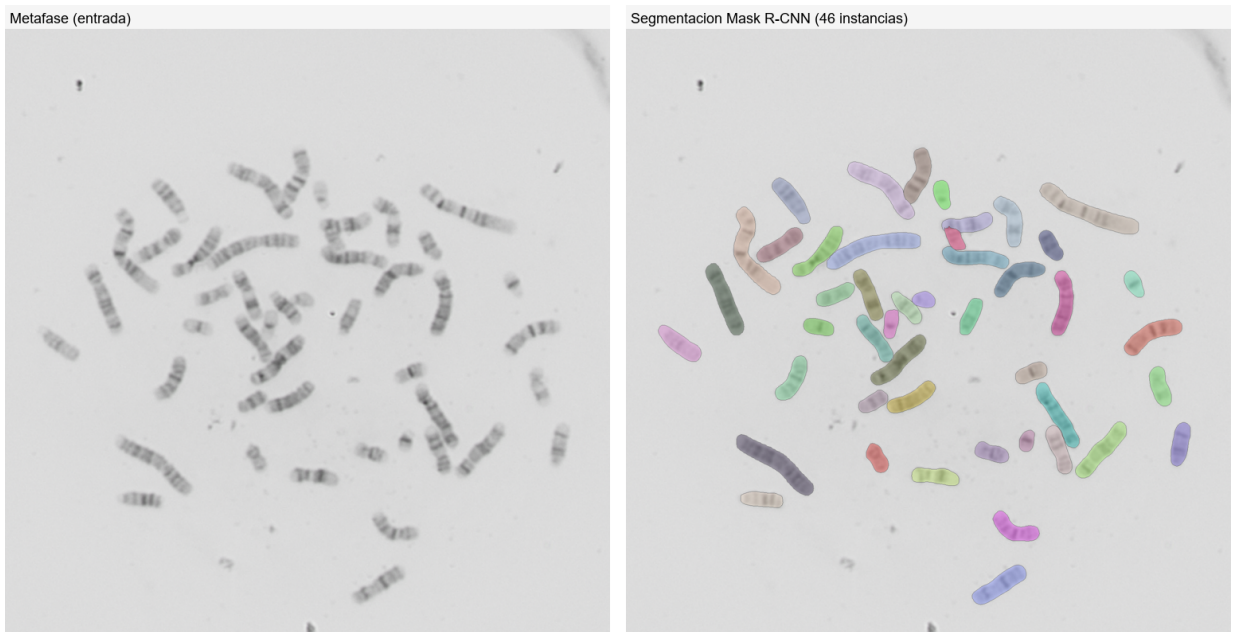


Figura 2: Ejemplo de segmentación: imagen de metafase (izquierda) y máscaras de instancia predichas por Mask R-CNN (derecha). El modelo separa correctamente cromosomas en contacto y superpuestos.

3.3. Extracción y registración

La etapa de extracción produce, por cada instancia segmentada, un recorte de 224×224 con el cromosoma aislado, rotado a vertical y escalado preservando su tamaño relativo (Figura 1), dejándolo en una pose canónica homogénea para las etapas siguientes.

3.4. Clasificación

La Tabla 4 compara el desempeño de las configuraciones exploradas sobre el conjunto de test (2874 recortes). El *backbone* resulta determinante: VGG16 (85–87 %) supera ampliamente a ResNet-50 (≈ 39 %), cuyo bajo rendimiento se atribuye al *fine-tuning* parcial de sus capas de *Batch Normalization* (cuya estadística se degrada al descongelarlas) y a un entrenamiento más corto. La atención SE por sí sola aporta poco (la versión v1 incluso descendió a 81 % por *callbacks* demasiado agresivos), pero, bien ajustada, supera al *baseline*. La mayor ganancia proviene de la

regularización y el balanceo de clases: el modelo final alcanza una **exactitud del 87 %** (F1 macro 0.86), el mejor resultado del conjunto.

Cuadro 4: Desempeño de los clasificadores sobre el conjunto de test (2874 recortes).

Modelo	Exactitud	F1 macro	F1 ponderado
ResNet-50	≈ 0.39	≈ 0.35	—
VGG16 (baseline)	0.85	0.85	0.85
VGG16 + SE v1	0.81	0.80	0.81
VGG16 + SE v2	0.86	0.86	0.86
VGG16 final	0.87	0.86	0.87

El entrenamiento del modelo final se detuvo por *early stopping* restaurando los pesos de la mejor época; sus curvas de exactitud y pérdida (Figura 3) muestran convergencia estable con un moderado sobreajuste (exactitud de entrenamiento $\approx 99\%$ frente a $\approx 87\%$ de validación). El análisis por cromosoma (Tabla 6, Anexo A) revela que el error residual se concentra en los cromosomas pequeños de morfología similar (16 a 22), la parte históricamente más difícil del cariotipado; el cromosoma Y (la clase minoritaria, con solo 24 muestras de test) es el de menor desempeño (F1 = 0.69) pese al balanceo por pesos.

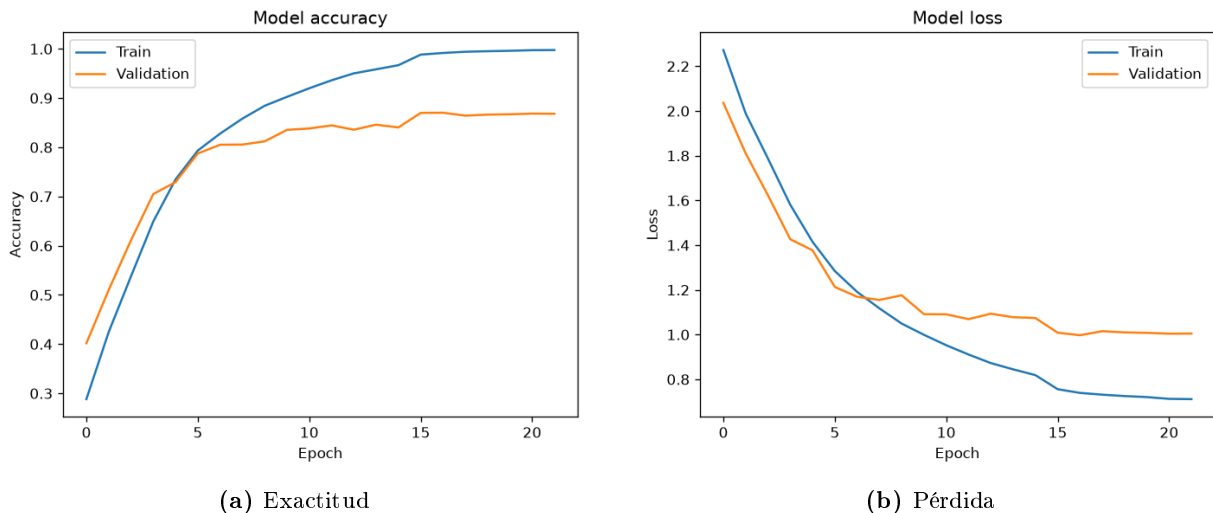


Figura 3: Curvas de entrenamiento y validación del clasificador final (VGG16 + SE + `class_weight` + `label_smoothing` + L_2).

3.5. Cariograma

Integrando todas las etapas, el pipeline arma el cariograma de forma automática. La Figura 4 muestra un ejemplo sobre una metafase de test: partiendo de la segmentación de referencia (para aislar la calidad de la clasificación y del ensamblado), los 46 cromosomas se rectifican, clasifican y ordenan en la grilla estándar, produciendo un cariotipo femenino normal (46,XX); en esa imagen la clasificación acierta el 97,8% de los cromosomas. El flujo completamente automático, con la segmentación predicha, se muestra en la Figura 1, donde los errores residuales de segmentación y clasificación se reflejan en el conteo por par. El desempeño varía por imagen y en promedio se mantiene en línea con la exactitud del clasificador reportada en la Sección 3.4. Para su uso en la interfaz, el clasificador se exporta a ONNX y corre con `onnxruntime`, sin necesidad de TensorFlow (misma estrategia que la segmentación con TorchScript), verificando la paridad de salidas con el modelo original.

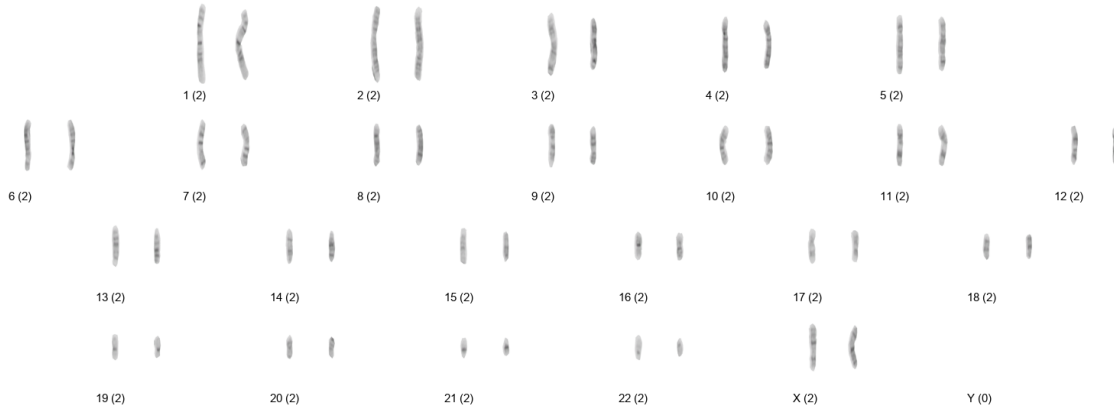


Figura 4: Cariograma ensamblado automáticamente por el pipeline a partir de una metafase de test. Cada celda muestra el par de homólogos rectificados, con el número de cromosoma y el conteo de instancias. Los cromosomas sexuales (X, Y) se ubican al final; el conteo por par permite marcar anomalías numéricas.

4. Conclusiones

Se desarrolló un pipeline de cariotipado automático que integra pre-procesamiento caracterizado cuantitativamente, segmentación de instancias con Mask R-CNN, extracción de características con registración por PCA, clasificación de cada cromosoma y ensamblado del cariograma, todo accesible desde una interfaz gráfica que visualiza el procesamiento paso a paso.

Comparación con el estado del arte. La segmentación desarrollada (97,8 % de AP@0.5) es competitiva con la de Wang *et al.* [2] (módulo *ChrRender*, 96,65 %–96,81 %) e incluso ligeramente superior, según se resume en la Tabla 5. Debe notarse que aquí se resuelve la segmentación de una única clase (cromosoma), mientras que parte de la dificultad del estado del arte proviene de escenarios adicionales; aun así, el resultado ubica a la etapa de segmentación en un nivel de estado del arte. En clasificación, el modelo final alcanza un 87 % de exactitud, por debajo del 94 %–95 % del módulo ChrNet4 de Wang *et al.* [2]. La diferencia es esperable: el modelo del estado del arte incorpora aumentación de datos y un diseño específico para el problema, mientras que el clasificador aquí desarrollado se entrenó sin *data augmentation* y sobre un conjunto con marcado desbalance de clases; el margen de mejora se concentra en los cromosomas de tamaño similar y en la clase minoritaria.

Cuadro 5: Comparación con el estado del arte [2].

	Segmentación (AP@0.5)	Clasificación (exactitud)
Wang <i>et al.</i> [2]	96.7–96.8	94–95
Este trabajo	97.8	87

Posibles mejoras. Se identifican dos líneas principales de trabajo:

- **Ablación del preprocesamiento.** Evaluar los modelos con la imagen cruda frente a la preprocesada, midiendo AP en segmentación y exactitud en clasificación sobre el conjunto de test. Esto convertiría en un resultado cuantitativo la decisión de operar sobre la imagen cruda: permitiría medir cuánto mejora o empeora cada etapa al preprocesar y a partir de qué intensidad de filtrado empieza a notarse la pérdida del bandeo G.
- **Mejorar la clasificación.** El error residual se concentra en los cromosomas pequeños de morfología parecida (16 a 22) y en la clase minoritaria Y, que aporta muy pocas muestras.

Se proponen dos vías complementarias: aumentación de datos (rotaciones, deformaciones elásticas y variaciones de intensidad) para exponer al modelo a más variabilidad sin recolectar imágenes nuevas, y balancear el conjunto sumando o replicando muestras de las clases minoritarias para que el clasificador deje de sesgarse hacia las mayoritarias.

En síntesis, el trabajo integra un flujo automático de cariotipado con una segmentación de nivel de estado del arte, una registración que estandariza los cromosomas y un clasificador que alcanza un 87 % de exactitud, y aporta una interfaz que hace tangible cada etapa del procesamiento de imágenes involucrado.

Referencias

- [1] D. You, P. Xia, Q. Chen, M. Wu, S. Xiang, and J. Wang, “AutoKary2022: A large-scale densely annotated dataset for chromosome instance segmentation,” in *IEEE International Conference on Multimedia and Expo (ICME)*, 2023, arXiv:2303.15839.
- [2] C. Wang, L. Yu, J. Su, J. Shen, V. Selis, C. Yang, and F. Ma, “Fully automatic karyotyping via deep convolutional neural networks,” *IEEE Access*, vol. 12, pp. 46 081–46 092, 2024.
- [3] K. He, G. Gkioxari, P. Dollár, and R. Girshick, “Mask R-CNN,” in *IEEE International Conference on Computer Vision (ICCV)*, 2017, pp. 2961–2969.
- [4] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.
- [5] T.-Y. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan, and S. Belongie, “Feature pyramid networks for object detection,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 2117–2125.
- [6] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, “ImageNet: A large-scale hierarchical image database,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2009, pp. 248–255.
- [7] K. Simonyan and A. Zisserman, “Very deep convolutional networks for large-scale image recognition,” in *International Conference on Learning Representations (ICLR)*, 2015.
- [8] J. Hu, L. Shen, and G. Sun, “Squeeze-and-excitation networks,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 7132–7141.
- [9] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick, “Microsoft COCO: Common objects in context,” in *European Conference on Computer Vision (ECCV)*, 2014, pp. 740–755.
- [10] J. Immerkær, “Fast noise variance estimation,” *Computer Vision and Image Understanding*, vol. 64, no. 2, pp. 300–302, 1996.
- [11] A. Buades, B. Coll, and J.-M. Morel, “A non-local algorithm for image denoising,” *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, vol. 2, pp. 60–65, 2005.
- [12] K. Zuiderveld, “Contrast limited adaptive histogram equalization,” in *Graphics Gems IV*. Academic Press, 1994, pp. 474–485.
- [13] Y. Wu, A. Kirillov, F. Massa, W.-Y. Lo, and R. Girshick, “Detectron2,” <https://github.com/facebookresearch/detectron2>, 2019.
- [14] A. Paszke, S. Gross, F. Massa *et al.*, “PyTorch: An imperative style, high-performance deep learning library,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

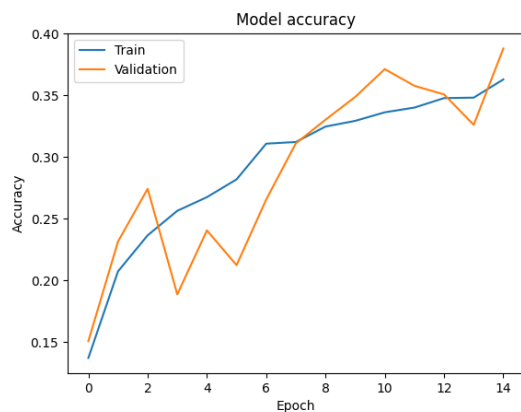
- [15] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in *International Conference on Learning Representations (ICLR)*, 2015.
- [16] I. T. Jolliffe and J. Cadima, “Principal component analysis: A review and recent developments,” *Philosophical Transactions of the Royal Society A*, vol. 374, no. 2065, 2016.
- [17] Streamlit Inc., “Streamlit: A faster way to build and share data apps,” <https://streamlit.io>, 2024.
- [18] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, “Image quality assessment: From error visibility to structural similarity,” *IEEE Transactions on Image Processing*, vol. 13, no. 4, pp. 600–612, 2004.

A. Anexo: detalle de la clasificación

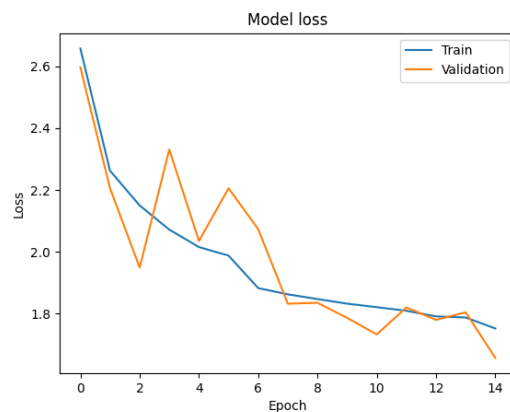
La Tabla 6 presenta el reporte de clasificación por clase del modelo final. Las Figuras 5–8 muestran las curvas de entrenamiento del resto de los modelos comparados.

Cuadro 6: Reporte de clasificación por cromosoma del modelo final (VGG16 + SE + `class_weight` + `label smoothing` + L_2). Las clases 23 y 24 del dataset corresponden a los cromosomas sexuales X e Y, respectivamente.

Cromosoma	Precisión	Recall	F1	Soporte
1	0.98	0.98	0.98	132
2	0.95	0.95	0.95	128
3	0.98	0.90	0.94	127
4	0.86	0.85	0.86	126
5	0.90	0.91	0.90	124
6	0.90	0.86	0.88	127
7	0.99	0.87	0.93	128
8	0.85	0.90	0.87	122
9	0.86	0.82	0.84	129
10	0.81	0.89	0.85	129
11	0.91	0.94	0.92	128
12	0.97	0.94	0.95	126
13	0.92	0.88	0.90	122
14	0.91	0.87	0.89	121
15	0.83	0.87	0.85	126
16	0.84	0.83	0.84	125
17	0.82	0.78	0.80	125
18	0.84	0.77	0.81	119
19	0.71	0.75	0.73	116
20	0.73	0.87	0.80	123
21	0.92	0.92	0.92	124
22	0.78	0.83	0.81	123
X (23)	0.81	0.83	0.82	100
Y (24)	0.68	0.71	0.69	24
Exactitud		0.87		2874
<i>Promedio macro</i>	0.87	0.86	0.86	2874
<i>Promedio ponder.</i>	0.87	0.87	0.87	2874

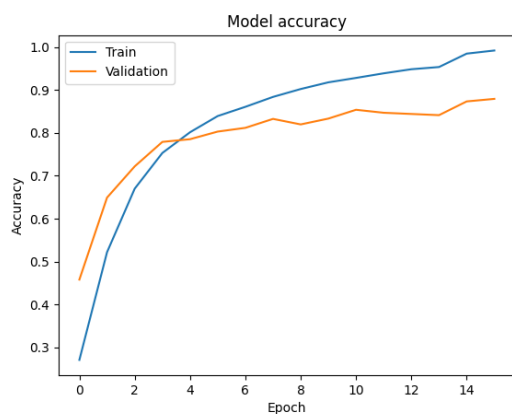


(a) Exactitud

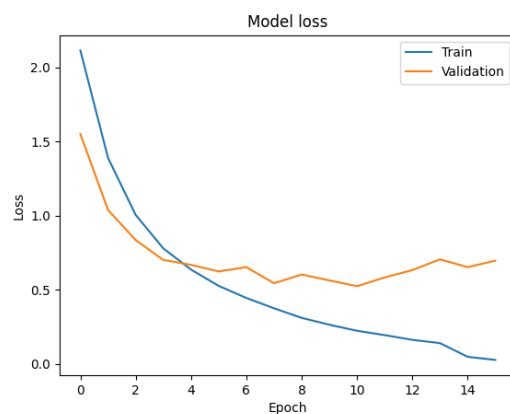


(b) Pérdida

Figura 5: ResNet-50: curvas de entrenamiento y validación.

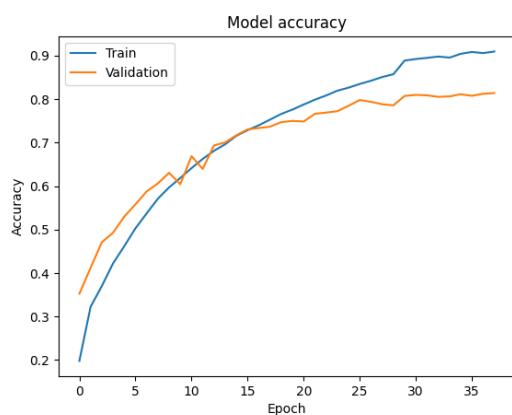


(a) Exactitud

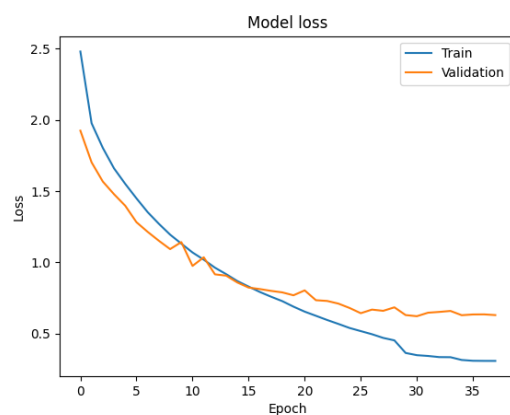


(b) Pérdida

Figura 6: VGG16 (baseline): curvas de entrenamiento y validación.

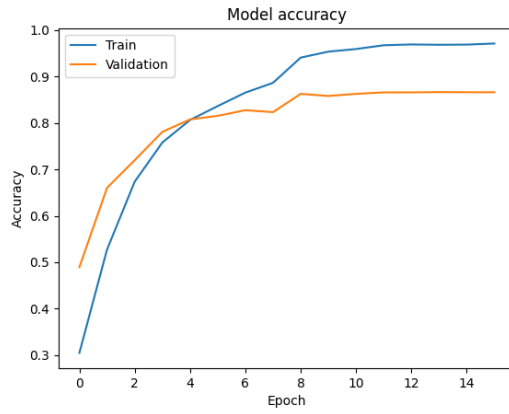


(a) Exactitud

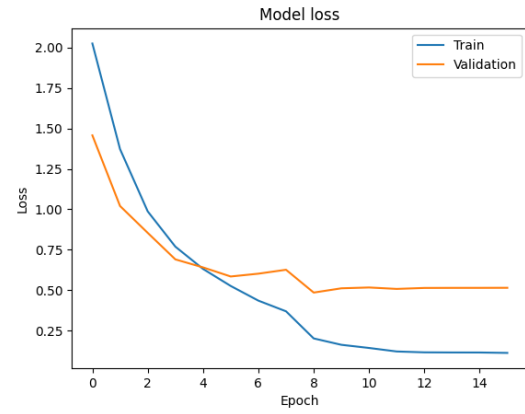


(b) Pérdida

Figura 7: VGG16 + SE v1: curvas de entrenamiento y validación.



(a) Exactitud



(b) Pérdida

Figura 8: VGG16 + SE v2: curvas de entrenamiento y validación.